

Publication-Oriented Research and System Design Roadmap

Repair, Don't Regenerate

From a Weather-Aware Itinerary Generator to an Inspectable,
User-Controlled, Solver-Verified Repair System

Research roadmap, preliminary paper design, mathematical formulation,
repository-specific implementation pipeline, and evaluation protocol

Yit Xiaang Ztang

University of Minnesota

June 30, 2026

Internal research planning document

Executive Summary

Primary research direction

Convert the existing California weather-aware route generator into an inspectable, user-controlled, solver-verified itinerary repair system. The central research object is not a newly generated itinerary; it is a persistent traveler-approved parent plan, an explicit disruption or edit, and the smallest certified child plan that preserves traveler-owned commitments.

The repository is already stronger than the original course prototype. It contains reusable data enrichment, hierarchical city/day allocation, Gurobi and heuristic route solvers, route alternatives, hotel/base decisions, artifact metadata, static dashboards, and a newly landed Phase 0 evidence foundation. The June 30 main branch now includes a clean-clone California snapshot, DatasetBundle validation, route-leg provenance contracts, PlannerRun and PlanArtifact records, independent eligibility reports, road-cache tooling, and strict comparison gates. These changes solve part of the earlier reproducibility and routing problems, but they do not yet implement parent-plan-aware optimization, ownership semantics, progressive repair, a thin authoritative pipeline runner, or evidence-grounded counterfactual explanations.

The recommended first paper is IUI-first: its main contribution should be a mixed-initiative optimization protocol that exposes constraint ownership, lets travelers specify the meaning and protection strength of edits, performs progressive local repair, and validates the final displayed plan independently. The user-study design should remain CHI-compatible by measuring plan ownership, mental-model accuracy, calibrated reliance, perceived control, and qualitative reactions to unwanted change. A transportation-oriented package can be prepared first by emphasizing disruption scenarios, route stability, repair quality, and validation; the IUI and CHI versions should add progressively richer interaction and human evidence.

The mathematical recommendation is a decomposed repair architecture. A master model decides POI retention, day assignment, lodging, ownership-dependent relaxations, and which parts of the parent plan may change. A day-route subproblem decides order and timing for affected days. An OSRM-backed routing service supplies road-valid time and distance matrices, preferably through a pinned local Docker deployment. An independent evaluator recomputes feasibility and certificate status from the final artifact. Sequential lexicographic solves are the primary implementation because each priority can be inspected and fixed before the next objective is optimized.

- Priority 0: minimize or forbid hard-constraint relaxations.
- Priority 1: minimize ownership-weighted plan changes.
- Priority 2: minimize risk, travel burden, and estimated cost.
- Priority 3: maximize destination utility, diversity, and preliminary scenic preference fit.

Two deliverables accompany this roadmap. This PDF is the human-facing research and preliminary system-design document. A separate Markdown technical specification translates the design into dependency-ordered work packages with exact interfaces, allowed files, invariants, failure behavior, tests, and acceptance criteria for Codex-assisted implementation.

Decision	Adopted direction
Primary framing	IUI-first mixed-initiative optimization; CHI-compatible human study; transportation evidence package first.
Core method	Decomposed repair master, day-route subproblems, routing service, independent validator, and sequential lexicographic objectives.
Data and infrastructure	Mandatory catalog/context/run separation; provider-neutral lodging taxonomy; credential-gated APIs; local OSRM Docker.
Scope control	Six disruptions in the benchmark; preliminary corridor-level scenic preference; edge-level OSMnx routing remains an extension.

Contents

1. Scope, evidence base, and decision record
 2. Current GitHub audit and Phase 0 status
 3. Research problem and novelty boundary
 4. Working thesis, contributions, and venue strategy
 5. Mixed-initiative interaction and traveler authority
 6. Formal objects, constraint ownership, and plan differences
 7. Decomposed mathematical repair formulation
 - 8-23. Detailed solutions for data, notebook, hotel, and routing Problems 1-8
 24. Repository-specific implementation architecture
 25. Six-disruption repair benchmark
 26. Algorithmic evaluation and statistical analysis
 27. IUI-first, CHI-compatible user-study protocol
 28. Preliminary scenic-routing implementation
 29. Dependency-ordered development roadmap
 30. Preliminary paper/system-design outline
 31. Claims, risks, and final recommendation
- Appendices: notation, schemas, repository audit, provider matrix, test matrix, references

1. Scope, Evidence Base, and Decision Record

This report integrates the publication-oriented roadmap, the traveler-facing research-direction report, the dataset and system-integration report, the detailed repair synthesis, and a fresh audit of the current GitHub main branch. It is intentionally both conservative and implementation-oriented: every feature is classified as implemented, partially implemented, or planned, and every planned research claim is connected to a concrete code path, artifact, or evaluation requirement.

The uploaded publication roadmap already identifies the strongest contribution as visible constraint ownership, graded edit semantics, progressive minimal repair, immutable plan lineage, independent validation, and evidence-grounded explanations. The research-direction report adds the interaction details needed for an IUI or CHI paper: direct manipulation is ambiguous, users must choose whether an edit is locked, strong, flexible, or test-only, and consequence previews must appear before commitment. The earlier dataset audit adds the scientific-data requirements: observed, curated, imputed, simulated, derived, and optimizer-produced values must not be silently mixed.

Decision criterion

A feature belongs in the first paper when it reuses the existing project, strengthens a narrow novelty claim, admits objective and human evaluation, and does not create a new data-collection problem that dominates the repair contribution.

Input	Main contribution to this report
Publication roadmap	Novelty boundary, formal repair thesis, evaluation metrics, data audit, and publication strategy.
Research-direction report	Traveler journey, edit semantics, direct manipulation, impact previews, explanation needs, and user-study questions.
System-integration report	Problems 1-8, snapshot design, hotel/routing provenance, notebook migration, and security.
Current GitHub main	What is already implemented, what recently landed in Phase 0, and exact modules that must change next.
Current official documentation	Venue dates, hotel/provider capabilities, routing deployment choices, and credential policies.

2. Current GitHub Audit and Phase 0 Status

The repository audit was performed against the current main branch on June 30, 2026. The immediately preceding commit is 482cbaefc98c7b366df3ae75fae5cf91a17feb62; the current head adds the Phase 0 research foundation. Because the connector did not expose the current head SHA directly, Appendix C records per-file blob hashes for the audited files. This avoids falsely claiming a commit identifier that was not retrieved.

The README still names the production notebook as the primary execution path and describes a 228-candidate California demo, saved 7/9/12-day routes, 33 route records, and 97 dashboard validation checks. It also clearly states the present limitations: the browser dashboard is static, preview controls do not rerun Gurobi, generated artifacts can become stale, live sources may fall back to caches, and congestion remains a proxy.

2.1 What Phase 0 already implemented

- A committed data/snapshots/california_v1 clean-clone foundation with entity, observation, feature, provenance, hotel, weather, route, and audit tables.
- DatasetBundle and DatasetValidationReport contracts plus load_dataset_bundle() and validate_dataset_bundle().
- catalog_snapshot_id, context_snapshot_id, refresh_policy, and run_id in configuration and artifact metadata.
- RouteLegResult and RouteResult with explicit geometry, distance, duration, provider, fallback, road_validated, and evaluation_eligible fields.

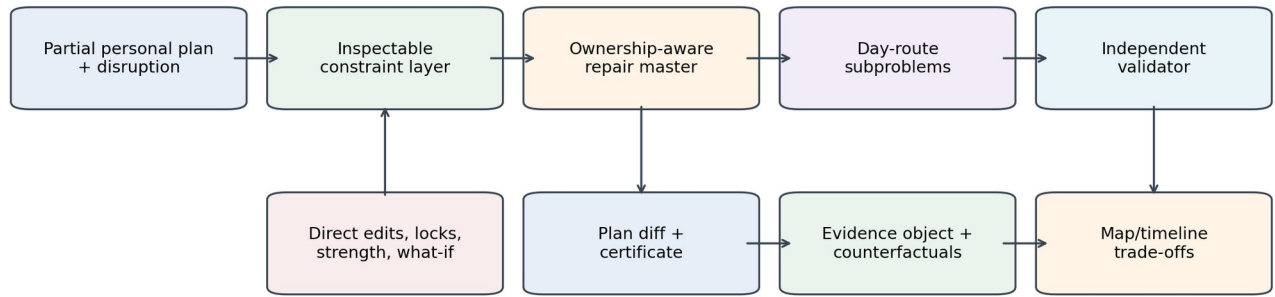
- PlannerRun, PlanArtifact, and ResearchEvaluationReport contracts that separate planner attempts, immutable plan content, and independent comparison eligibility.
- Phase 0 evidence exports, route-cache audits, strict validation scripts, and a default local OSRM endpoint policy.

2.2 What remains missing for the paper

- OwnedConstraint, EditRequest, DisruptionSnapshot, RepairNeighborhood, RepairOutcome, and smallest-relaxation schemas.
- A solver that receives a parent PlanArtifact and explicitly optimizes deletion, addition, day movement, time shift, order, lodging, and road changes.
- Progressive repair radii that freeze unaffected days and expand only after certified infeasibility.
- A package-level pipeline_runner.py used identically by CLI, notebook, tests, and future service endpoints.
- A generalized plan_diff.py, final-plan evaluator, counterfactual explanation engine, and evidence-linked natural-language layer.
- Provider-independent lodging classification and credential-gated hotel metadata/rate adapters.
- Local OSRM Docker assets and a single routing client that supplies the optimizer, map, evaluation, and audit from the same records.

Module	Current evidence	Next action
schemas.py	Only simple trip, POI, city, budget, route, and experiment dataclasses.	Retain compatibility; move research schemas into focused plan/repair modules.
data/snapshot.py	Loads and validates the new California snapshot.	Separate context directories, add schema versions, manifest file hashes, and materialized optimizer views.
research_artifacts.py	PlannerRun, PlanArtifact, ResearchEvaluationReport exist.	Extend PlanArtifact with owned constraints, day objects, hotels, route edges, certificate and provenance links.
multi_objective_route.py	Single-day x/y/u MTZ route with time, cost, detour, weather, diversity.	Turn into day-route subproblem; remove parent-repair terminology from current heuristic.
hierarchical_gurobi.py	Generates/scored city-day candidates; no parent plan.	Add repair master or wrapper that freezes parent structure and exposes change variables.
route_gurobi_oracle.py	Delegates to multi_objective_route; old implementation remains unreachable after early return.	Delete dead code after regression tests; expose one typed solver interface.
experiment_runner.py	Notebook-oriented orchestration and flat production artifacts.	Split pipeline orchestration, benchmark execution, and artifact export.
map_renderer.py	Sanitizes stale notebook context before rendering.	Consume canonical plan artifacts instead of mutating notebook dictionaries.

Publication-ready mixed-initiative repair loop



The traveler reviews interpretation, consequences, and evidence before accepting a child plan.

Figure 1. Target publication-ready control loop.

3. Research Problem and Novelty Boundary

Travelers rarely begin from an empty page. They bring saved places, reservations, preferred roads, fixed dates, and personal rules. Existing AI planners often replace this work with a complete regeneration. The research problem is therefore not merely finding a feasible route after a disruption; it is preserving the parts of an existing plan that the traveler owns, exposing when preservation becomes impossible, and certifying the smallest acceptable change.

Several neighboring ideas are already established. TripTide evaluates itinerary revision under disruptions and intent preservation. iTIMO formalizes itinerary modification operations. TTG and related systems translate language into symbolic planning inputs. OptiChat and explainable-optimization work support infeasibility, sensitivity, and counterfactual dialogue. Scenic-routing research already assigns pleasantness or beauty to routes. The paper must not claim any one of these components as entirely new.

Defensible novelty statement

We study how visible constraint ownership, graded edit semantics, sequential lexicographic optimization, and progressive local repair can preserve a traveler-approved itinerary under disruptions, and how independent certification and evidence-linked visual explanations affect understanding, control, and calibrated reliance.

Claim	Status
“We repair a trip after bad weather.”	Insufficient: disruption-aware revision already exists.
“An LLM converts requests to constraints.”	Insufficient: language-to-symbolic planning already exists.
“Users drag stops on a map.”	Insufficient: direct manipulation and critique-based planning already exist.
“An LLM explains a solver.”	Insufficient without structured evidence and verification.
“We choose a scenic route.”	Insufficient without a stronger integration with ownership and repair.
Full ownership-repair-certificate-explanation loop	Defensible as a system, protocol, and evaluation contribution.

4. Working Thesis, Contributions, and Venue Strategy

Working thesis

Travelers should not be forced to choose between opaque AI regeneration and manual repair. A solver-grounded mixed-initiative system can preserve user-owned plan elements, expose the consequences of edits, and adapt only the smallest necessary part of an itinerary while maintaining verifiable feasibility.

4.1 Proposed contribution bundle

1. A constraint-ownership representation that records origin, strength, scope, confirmation state, confidence basis, and relaxation cost.
2. A decomposed progressive-repair algorithm with sequential lexicographic priorities and explicit plan-change variables.
3. An immutable evidence protocol linking parent plan, disruption, edit, solver stages, routing context, child plan, plan diff, independent evaluation, and explanation.
4. An interaction design for constraint inspection, edit-strength confirmation, before/after route comparison, consequence preview, and counterfactual exploration.
5. A computational benchmark and controlled user study measuring feasibility, preservation, plan quality, comprehension, control, and calibrated reliance.

4.2 Venue sequence

Target	Best framing	Required evidence gate
TRB-oriented first package	Disruption repair, route stability, road validation, scenario benchmark, solver performance.	Certified benchmark, strong transportation baselines, no unsupported human-outcome claims.
IUI 2027 primary	Mixed-initiative optimization, inspectable constraints, direct edits, evidence-grounded explanations.	Working interactive prototype, computational study, controlled user study.
CHI 2027 extension	Plan ownership, agency, consequence preview, mental models, calibrated reliance, qualitative design implications.	Stronger study/qualitative analysis and polished interaction.

The official IUI 2027 schedule lists paper abstract registration on August 13 and full papers on August 20, 2026. The official CHI 2027 papers page lists September 10, 2026 for the full-paper deadline. The project should therefore use dependency gates rather than a fixed weekly schedule: a transportation-ready benchmark first, an IUI interaction package immediately afterward, and a CHI-strength human-centered version only if the study evidence is mature. The user-selected August 1 TRB target should be treated as an internal readiness target until the current Annual Meeting call is verified.

5. Mixed-Initiative Interaction and Traveler Authority

The interaction model should preserve the strongest ideas from the research-direction report: the system first displays its interpretation, the traveler corrects it, a feasible working plan is selected, and later edits are converted into explicit solver semantics. The LLM proposes; the user confirms; the solver optimizes; the verifier certifies; the interface explains.

5.1 Edit semantics are not inferable from drag-and-drop alone

Moving Yosemite to Day 4 could mean a hard day assignment, a strong preference, a requirement to keep Yosemite with flexible timing, or a temporary what-if test. The interface must ask how strongly the edit should be protected. A direct manipulation event is therefore not itself a constraint; it is evidence from which an EditRequest is constructed and confirmed.

User control	Formal meaning	Default solver treatment
Lock it	The requested value may not change.	Hard equality or forbidden change variable.
Booked / prepaid	Change only after explicit confirmation.	Very high relaxation penalty and separate relaxation variable.
Strong preference	Change only for feasibility or major improvement.	High ownership-dependent edit cost.
Flexible suggestion	May change for a better plan.	Moderate or low penalty.
Test only	Generate a sandbox child without changing accepted parent.	Separate run; never overwrites accepted plan.

5.2 Consequence preview

Before committing an edit, the system should run either a fast local feasibility check or a sandbox repair and show the expected effects: protected items retained, changed days, extra driving, hotel impact, risk shift, and certificate state. The preview must distinguish estimated consequences from fully certified consequences.

Proposed edit: Move Yosemite to Day 4 (strong preference)

Expected benefits

- + Lower outdoor weather exposure on Day 4
- + Santa Barbara booking remains protected

Expected costs

- 42 additional road-validated driving minutes
- Monterey activity moves to Day 5
- Day 4 has 8 minutes of remaining drive-time slack

Status: feasible sandbox repair; not yet accepted as the working plan

6. Formal Objects, Constraint Ownership, and Plan Differences

6.1 Core immutable objects

Object	Required role
PlanArtifact	Immutable accepted or candidate itinerary, including stops, days, hotels, route legs, constraints, snapshots, source run, and content hash.
OwnedConstraint	Rule with origin, strength, scope, confirmation, evidence, and relaxation policy.
EditRequest	Confirmed operation, target, desired value, strength, and test-only state.
DisruptionSnapshot	Versioned weather, road, hotel, attraction, or traveler-capacity change.
RepairNeighborhood	Explicit editable days, stops, hotels, and edges at a given radius.
PlannerRun	One solver attempt, including failed/fallback stages and objective results.
PlanDiff	Typed additions, deletions, moves, reorderings, hotel changes, road changes, and unchanged regions.
EvaluationReport	Independent final-artifact feasibility, grounding, warning, and comparison eligibility.
ExplanationEvidence	Structured why/why-not/what-if records linked to constraint, diff, route, and counterfactual IDs.

6.2 Ownership-dependent weights

$\text{rho}(c) \in \{\text{infinity}, \text{rho_booked}, \text{rho_strong}, \text{rho_weak}, \text{rho_solver}, 0\}$

A user-locked rule is enforced as a hard constraint. Booked items receive a separate relaxation variable and very high priority. Solver-selected items have the lowest preservation cost. Test-only edits do not modify the accepted parent.

Origin / strength	Illustrative rho	Interpretation
user_locked	hard	Change prohibited unless the user changes the rule.

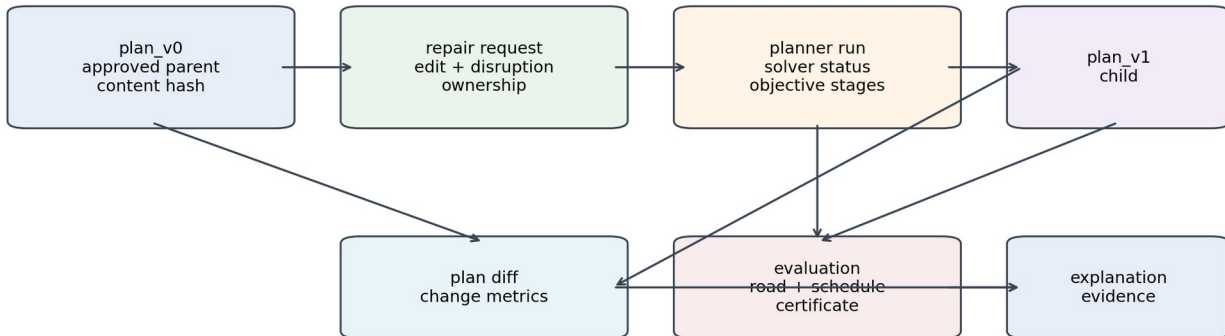
Origin / strength	Illustrative rho	Interpretation
booked_or_prepaid	1000	Relax only in a dedicated stage with explicit confirmation.
user_strong	100	Preserve before contextual quality improvements.
user_weak	10	Preserve when inexpensive.
solver_selected	1	Easy to replace or remove.
test_only	0 in accepted lineage	Applies only to sandbox child run.

6.3 Plan-difference decomposition

$$C_Delta(pi, pi0) = C_del + C_add + C_day + C_time + C_order + C_hotel + C_road + C_relax$$

The decomposition is essential for both optimization and explanation. A single “repair score” cannot tell a reviewer or traveler whether the system preserved bookings but reordered optional stops, or instead changed the hotel while keeping the same POIs. Each component must be computed from typed variables and independently re-derived by `plan_diff.py`.

Immutable parent-child plan lineage



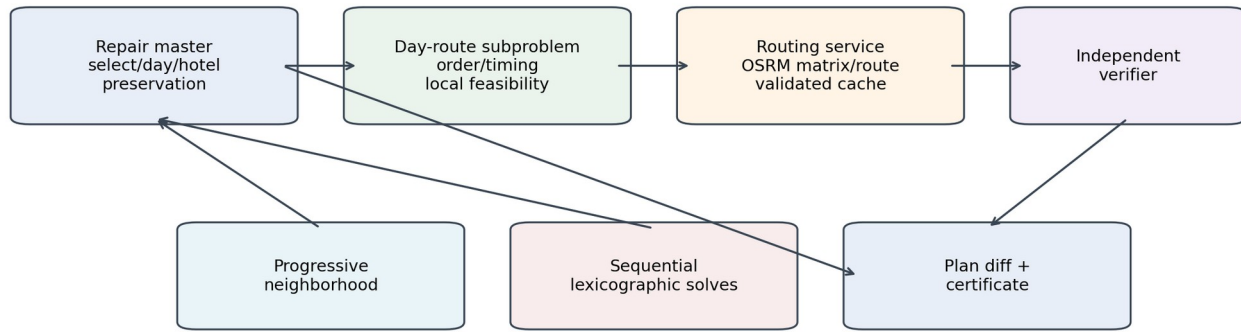
Any post-solve mutation creates a new child plan and invalidates the old certificate until re-evaluated.

Figure 2. Parent-child plan lineage and evidence objects.

7. Decomposed Mathematical Repair Formulation

The recommended formulation is decomposed rather than monolithic. The repair master decides inclusion, day assignment, overnight lodging, ownership relaxations, and the editable neighborhood. Day-route subproblems decide within-day order, arrival time, and road-leg feasibility. The routing service supplies consistent road times and geometry. The independent verifier recomputes the final displayed plan. This matches the repository's hierarchical structure and permits gradual migration from the current solvers.

Decomposed repair architecture



The master decides what may change; day-route solvers and routing services certify how it changes.

Figure 3. Decomposed repair master, day routes, routing service, and verifier.

7.1 Sets, indices, and parent-plan parameters

Symbol	Meaning
D	Trip days.
I	Candidate POIs, including all parent stops.
H	Candidate lodging entities.
A	Directed candidate arcs between depots, POIs, and hotels.
C	Owned constraints.
R	Repair radii or neighborhoods.
x^0_{id}	1 if POI i is assigned to day d in parent plan.
z^0_{hd}	1 if hotel h is used after day d in parent plan.
q^0_{ijd}	1 if parent route uses arc (i,j) on day d .
τ_{ij}	Road-valid travel time supplied by the routing context.
s_i	Visit duration.
$[a_i, b_i]$	Opening-time window.
u_i	POI utility under a fixed data snapshot.
$F_{id}(\omega)$	Feasibility of POI i on day d under disruption ω .
$\rho_i, \rho_h, \rho_{ij}$	Ownership-dependent change weights.

7.2 Master decision variables

Variable	Type	Meaning
x_{id}	binary	POI i is assigned to day d .
v_i	binary	POI i is selected somewhere in the child plan.
z_{hd}	binary	Hotel h is assigned after day d .
δ_i^-	binary	Parent POI i is deleted.
δ_i^+	binary	POI i is newly added.
m_{id}	binary	POI i moves from its parent day to d .
η_{hd}	binary	Hotel assignment differs from parent.
r_{ijd}	binary	Road/corridor arc differs from parent.
x_i^c	binary/continuous	Owned constraint c is relaxed.
θ_d	binary	Day d is unlocked by the repair neighborhood.

7.3 Selection and assignment constraints

$$v_i = \sum_{d \in D} x_{id} \quad \text{for all } i; \quad \sum_{d \in D} x_{id} \leq 1$$

A POI is assigned to at most one day. Required parent/user items can force $v_i = 1$; unavailable items force the corresponding $x_{id} = 0$.

$$x_{id} \leq F_{id}(\omega) \quad \text{for all } i, d$$

The disruption snapshot determines whether a POI-day assignment is feasible. For a weather deterioration this may block outdoor assignments; for an attraction closure it blocks the affected time window.

$$x_{id} = x_{id}^0 \quad \text{for all } d \text{ not in } N_r \text{ and all } i \text{ outside the editable neighborhood}$$

Progressive repair freezes every parent assignment outside the selected radius.

7.4 Typed plan-change variables

$$\delta_i^- \geq v_i^0 - v_i; \quad \delta_i^+ \geq v_i - v_i^0$$

$$m_{id} \geq x_{id} - x_{id}^0; \quad \eta_{hd} \geq z_{hd} - z_{hd}^0$$

For exact accounting, the implementation should either use equality-strengthening constraints or compute the final typed changes from the child PlanArtifact. The optimizer variables provide the objective; plan_diff.py provides the authoritative post-solve record.

7.5 Day-route subproblem

$$\sum_j y_{ijd} = x_{id}, \quad \sum_j y_{jid} = x_{id}$$

Each selected POI has one entering and one leaving arc on its day. The depot or hotel anchors provide the day start and end nodes.

$$T_{jd} \geq T_{id} + s_i + \tau_{ij} - M(1 - y_{ijd})$$

Arrival times propagate along selected road-valid arcs. Time windows impose $a_i x_{id} \leq T_{id} \leq b_i + M(1 - x_{id})$.

$$\sum_i s_i x_{id} + \sum_{(i,j)} \tau_{ij} y_{ijd} \leq B_d^{\text{time}}$$

The daily schedule budget uses the same route-leg times as the map and independent evaluator.

The existing `multi_objective_route.py` already contains x , y , order variables, MTZ subtour constraints, and time/cost/weather/diversity limits. It should be refactored into a typed `DayRouteSolver` that accepts a selected-day candidate set and a `RouteMatrix` rather than calculating geodesic travel internally.

7.6 Hotel continuity and lodging category preferences

$$\sum_{h \in H_d} z_{hd} = 1 \text{ for each overnight } d$$

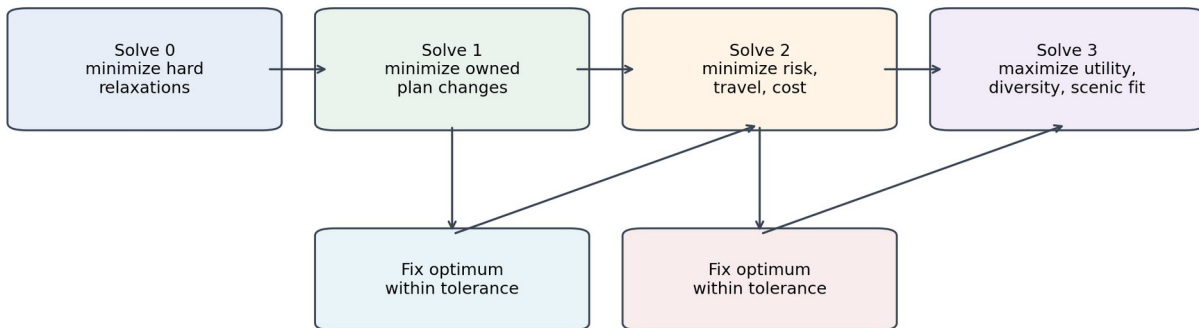
Exactly one lodging entity is selected for every overnight day, unless the scenario explicitly permits no lodging assignment.

$$z_{hd} \leq \text{category_allowed}(h, \text{traveler}); \quad \eta_{hd} \geq z_{hd} - z^0_{hd}$$

Lodging category is modeled independently of provider brand. A traveler may permit hotel, motel, hostel, guest house, bed-and-breakfast, apartment, vacation rental, private room, campground, cabin, or RV park. A listing is called “Airbnb” only when provider evidence identifies Airbnb; otherwise it remains a provider-neutral `vacation_rental` or `private_room` entity.

7.7 Sequential lexicographic objectives

Sequential lexicographic optimization



A marginal utility gain cannot justify a larger user-owned change.

Figure 4. Recommended sequential lexicographic solve order.

$$\text{Stage 0: minimize } L_0 = \sum_{c \in C_{\text{hard_relaxable}}} M_c x_{ic}$$

$$\text{Stage 1: minimize } L_1 = \sum_i \rho_i^- \Delta_i^- + \rho_i^+ \Delta_i^+ + \sum_{\{i,d\}} \rho_i^{\text{day}} m_{id} + \sum_{\{h,d\}} \rho_h \eta_{hd} + \sum_{\{i,j,d\}} \rho_{ij} r_{ijd}$$

$$\text{Stage 2: minimize } L_2 = \lambda_w \text{WeatherRisk} + \lambda_t \text{TravelTime} + \lambda_c \text{Cost} + \lambda_h \text{HotelSwitches}$$

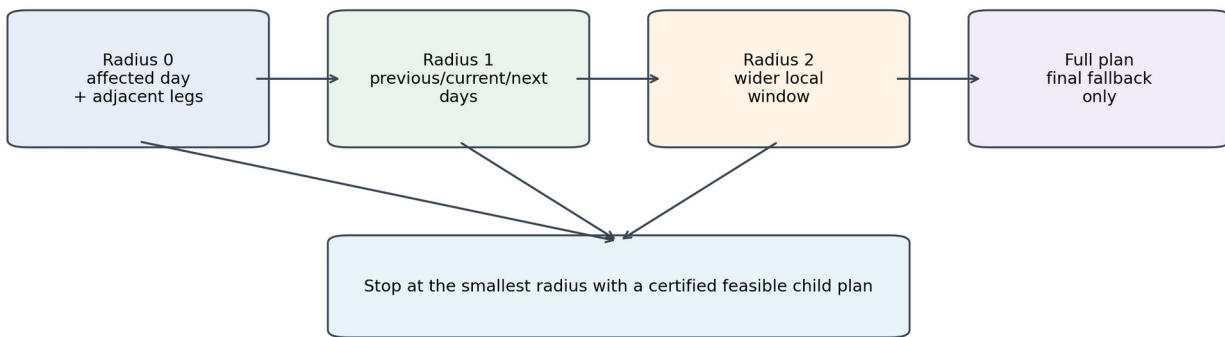
Stage 3: maximize $L3 = \sum_{i,d} u_i x_{id} + \lambda_{div} \text{Diversity}(x) + \lambda_{scenic} \text{ScenicFit}(q)$

After each stage, fix the optimum within a numerical tolerance before solving the next stage. This is more auditable than a single weighted sum because the report can show that no child plan with fewer important changes existed within the encoded model. Gurobi setObjectiveN may later replace the sequence when equivalent priority and tolerance behavior is verified by tests.

```
solve_stage_00
model.addConstr(L0 <= L0_star + tol0)
solve_stage_10
model.addConstr(L1 <= L1_star + tol1)
solve_stage_20
model.addConstr(L2 <= L2_star + tol2)
solve_stage_30
```

7.8 Progressive neighborhood algorithm

Progressive repair neighborhood



Every expansion is logged; unchanged distant days remain frozen.

Figure 5. Smallest certified neighborhood before full replanning.

```
for radius in request.allowed_radii: # [0, 1, 2, FULL]
    neighborhood = build_repair_neighborhood(parent, disruption, radius)
    model = build_repair_master(parent, request, neighborhood, data_context)
    candidate = solve_lexicographically(model)
    if candidate is None:
        continue
    routed = solve_affected_day_routes(candidate, routing_context)
    evaluation = independent_evaluator.evaluate(routed)
    if evaluation.comparison_eligibility == 'eligible':
        return certified_child(candidate, evaluation, radius)
return smallest_relaxation_diagnosis(...)
```

The stop rule is the smallest radius yielding a certified feasible child, not the first solver incumbent. Every attempt is retained as a PlannerRun, including infeasible and fallback attempts. This creates both an algorithmic stability definition and an explanation trace.

7.9 Formal preservation and quality metrics

$$\text{LockedPreservation} = (\# \text{ unchanged locked elements}) / (\# \text{ locked elements})$$

$$\text{UnaffectedDayPreservation} = (\# \text{ identical days outside disruption neighborhood}) / (\# \text{ days outside neighborhood})$$

$$\text{UtilityRegret} = U(\text{pi_full_reopt}) - U(\text{pi_repair})$$

$$\text{CertificateCoverage} = (\# \text{ final displayed plans independently eligible}) / (\# \text{ displayed plans})$$

$$\text{ExplanationEvidenceCoverage} = (\# \text{ explanation claims linked to valid evidence}) / (\# \text{ explanation claims})$$

7.10 Model boundary and implementation rationale

The decomposition is also a software boundary. The master may use only immutable catalog/context inputs and ownership policy; it may not fetch APIs or draw map geometry. The day-route solver consumes a route matrix and returns ordered, timed legs. The routing service owns provider calls and validation. The evaluator reconstructs the final plan independently rather than trusting solver flags.

This separation allows the current hierarchical and single-day solvers to migrate incrementally. Phase 0 artifact and routing contracts remain valid, while each new repair component can be tested against hand-built instances before the notebook or dashboard is changed.

The implementation should preserve two statuses at every boundary: mathematical feasibility under the encoded model and evidence eligibility for publication comparison. A feasible incumbent with approximate travel remains a useful demo, but it is not a road-validated research result.

Part II

Detailed Data and System Integration Solutions

The following sections preserve the detailed Problems 1-8 engineering plan while updating its interpretation: the current main branch has already landed a Phase 0 version of the snapshot, route-provenance, and research-artifact foundation. These sections are therefore both design requirements and migration checklists. “Implement” means complete and integrate the contract across the authoritative pipeline, not recreate files that already exist.

8. Problem 1: Missing Reproducible Fallback Dataset

The production notebook currently expects private Yelp files when available and otherwise references a fallback CSV under `results/outputs`. Because the dataset directory and generated output CSVs are ignored, that fallback cannot be guaranteed in a clean clone. The solution is to establish an explicit, immutable catalog snapshot as a first-class input rather than treating a prior run output as a source dataset.

8.1 Core rule: generated outputs are never source inputs

Input-output boundary

Files under `results/`, `runs/`, and dashboard export folders may be inspected or compared, but they must never be silently loaded as the authoritative attraction catalog for a later experiment. Reproducible source inputs belong under versioned data snapshots or explicitly configured external-data locations.

This rule prevents circular lineage: a route generated from one configuration cannot accidentally become the data input for another configuration. It also makes the difference between a source snapshot and a run artifact visible to reviewers and future contributors.

8.2 Two supported execution modes

Mode	Data behavior	Network behavior	Primary use
Reproducible benchmark mode	Load an exact catalog snapshot and an exact context snapshot	No implicit refresh; missing data causes a clear validation error	Paper experiments, regression tests, and supervisor review
Enrichment/build mode	Read configured external files and APIs, then write a new immutable snapshot	Network access is explicit and recorded in the manifest	Creating `california_v2`, expanding cities, or refreshing metadata
Approximate demo mode	Use the committed catalog but permit labeled context fallbacks	May use cached or approximate weather/routing context	UI demonstrations; excluded from final road-valid evaluation when required

8.3 Deterministic dataset resolution

```
def load_dataset_bundle(*, catalog_snapshot_id: str, context_snapshot_id: str | None,
    refresh_policy: str = "never") -> DatasetBundle:
    # 1. Resolve the exact catalog snapshot requested by ID.
    # 2. Verify its manifest, schema version, and file hashes.
    # 3. Resolve an exact context snapshot or create one only when explicitly requested.
    # 4. Never search results/outputs for a convenient prior CSV.
    # 5. Fail with an actionable error when required inputs are unavailable.
    ...
```

The default configuration should name `california\_v1`. A user may override that ID, but the loader must never choose a snapshot merely because it is the newest file on disk. Snapshot selection is part of the experiment definition.

8.4 Committed fallback snapshot

```
data/snapshots/california_v1/
├─ manifest.json
├─ poi_entities.parquet
├─ poi_observations.parquet
├─ poi_features.parquet
├─ feature_provenance.parquet
├─ hotels.parquet
├─ route_options.parquet
└─ source_audit.parquet
```

The committed snapshot should be based on redistributable open records and versioned curated annotations. Private Yelp data may be joined locally during snapshot construction, but the clean-clone snapshot must not require Yelp. A Yelp-enriched snapshot can remain local or publish only permitted derived aggregates.

8.5 Snapshot manifest contract

Manifest field	Purpose
snapshot_id and parent_snapshot_id	Immutable identity and lineage across catalog versions
created_at and created_by_pipeline	When and how the snapshot was built
code_commit and schema_version	Exact implementation and contract used
source registry entries	Provider, query scope, license, and redistribution policy
file paths and SHA-256 hashes	Detection of stale, edited, or incomplete files
row counts and coverage summary	Fast sanity check by city, category, and source
curated-annotation version	Traceability for must-go labels, priors, and manual corrections
contains_private_or_restricted_data	Prevents accidental publication of nonredistributable inputs

8.6 Build, freeze, and promote workflow

1. Build a candidate snapshot in a temporary staging directory from configured source adapters.
2. Validate schemas, required fields, duplicates, provenance coverage, and license metadata.
8. Generate the manifest and file hashes after all data files are finalized.
9. Run the clean-clone smoke test and the canonical California optimization test.
10. Promote the staging directory to a new immutable snapshot ID; never overwrite `california\_v1` in place.
11. Update the default configuration only after regression comparisons are accepted.

8.7 Acceptance criteria and tests

- A fresh clone with no private Yelp files can execute the canonical notebook and CLI using `california\_v1`.
- Deleting the complete `results/` directory does not change dataset loading behavior.
- Any edited snapshot file causes a manifest-hash failure before optimization begins.
- The run manifest records the exact catalog and context snapshot IDs.
- Live API access occurs only under an explicit refresh or build command.
- A missing required snapshot produces a clear remediation message instead of silently substituting another file.

9. Problem 2: Mixed Evidence Types in the Canonical POI Table

The current enriched catalog is convenient because it combines source identifiers, ratings, manually curated signals, heuristic scores, simulated values, context features, and final utility in one DataFrame. The same convenience weakens scientific traceability. The recommended solution is a layered data contract in which each value has one evidence class, one owner, and a reconstructable transformation path.

9.1 Five-layer scientific data contract

Layer	Contains	Write policy	Examples
Entity	Stable identity and cross-source identifiers	Changed only by entity-resolution workflow	POI ID, canonical name, coordinates, OSM/Wikidata/NPS IDs

Layer	Contains	Write policy	Examples
Observation	Values reported by one source at one time	Append or supersede; never overwrite another source	Yelp rating, OSM tags, NPS hours, Wikipedia description
Feature	Derived, imputed, curated, or simulated model inputs	Rebuilt by a versioned transformation	Nature score, duration prior, route fit, simulated demand proxy
Provenance	Lineage from feature to observations, annotations, and code	Mandatory for every optimizer-facing feature	Transformation name/version, source IDs, evidence kind
Decision	Optimizer selections, route assignments, and evaluation results	Written only inside run artifacts	Selected stop, day assignment, hotel choice, edit cost

9.2 Explicit evidence classification

```
class EvidenceKind(str, Enum):
    OBSERVED = "observed"
    CURATED = "curated"
    IMPUTED = "imputed"
    SIMULATED = "simulated"
    DERIVED = "derived"
    OPTIMIZER_OUTPUT = "optimizer_output"
```

Each feature record should identify exactly one primary evidence kind and may additionally reference supporting observations or annotations. For example, a hotel rate returned by LiteAPI is observed context, a city/type hotel price prior is curated or imputed, and a Bayesian utility score is derived. The final selected hotel is an optimizer output.

9.3 Materialized optimizer view

```
poi_entities
+ source observations
+ versioned features
+ context snapshot
+ traveler constraints
↓
build_optimizer_view(...)
↓
OptimizationDataset (read-only, validated, run-specific)
```

The optimizer may continue to use a wide table for performance and simple indexing. The important change is that this table becomes a generated materialized view, not the scientific source of truth. It should be rebuilt for each run from the catalog, context, and configuration and then stored with its manifest for debugging.

```
optimizer_data = build_optimizer_view(
    catalog=bundle.catalog,
    context=bundle.context,
    traveler_request=request,
    feature_set="repair_v1",
)
optimizer_data.validate_for_optimization()
```

9.4 Feature provenance contract

Field	Meaning
entity_id and feature_name	The entity and optimizer-facing value being explained
feature_value and unit	Typed value and interpretation

Field	Meaning
evidence_kind	Observed, curated, imputed, simulated, or derived
source_record_ids	Specific source observations used
annotation_ids	Manual inputs or priors used
transformation_name and version	Code path that produced the value
snapshot_id and context_snapshot_id	Stable and time-sensitive inputs
generated_at and code_commit	When and under which implementation it was computed
quality flags	Missing, fallback, stale, conflicting, or manually reviewed

9.5 Data-boundary enforcement

- Source adapters may write source observations but may not write utility, selection, or repair fields.
- Feature builders may read observations and annotations but may not alter the original source payload or entity identity.
- The optimizer receives only a validated `OptimizationDataset`; it must not call APIs or mutate catalog tables.
- Dashboard exporters read plan artifacts and provenance but may not recompute optimization features independently.
- Simulated or imputed values must include a method, version, and fallback reason.
- Optimizer-output columns are forbidden in committed entity, observation, and feature source tables.

9.6 Migration from the current enriched CSV

1. Inventory every current column and classify it by layer and evidence kind.
2. Extract identity fields into `poi\_entities` and provider-specific values into `poi\_observations`.
8. Move hard-coded must-go labels, category corrections, duration priors, and hotel priors into versioned annotation files.
9. Reimplement nature, utility, route-fit, and demand-proxy columns as versioned feature builders with provenance rows.
10. Create `build\_optimizer\_view()` that reproduces the current optimizer-required schema from the new layers.
11. Compare old and new optimizer views on the same snapshot and explain every material numerical difference.
12. Retire direct reads of `production\_enriched\_poi\_catalog.csv` once package tests and notebook execution use the new view.

9.7 Problem 2 acceptance criteria

- Every optimizer-facing field has a provenance record or an explicit exemption for a primary key.
- A reviewer can distinguish observed, curated, imputed, simulated, derived, and decision values without reading source code.
- No source observation is destroyed when two providers describe the same POI.
- The canonical optimizer view can be regenerated deterministically from the registered snapshots and configuration.
- Changing a feature transformation version produces a new feature set and does not silently rewrite the old snapshot.
- Explanations can cite the exact values and transformations used by the solver.

10. Target Architecture

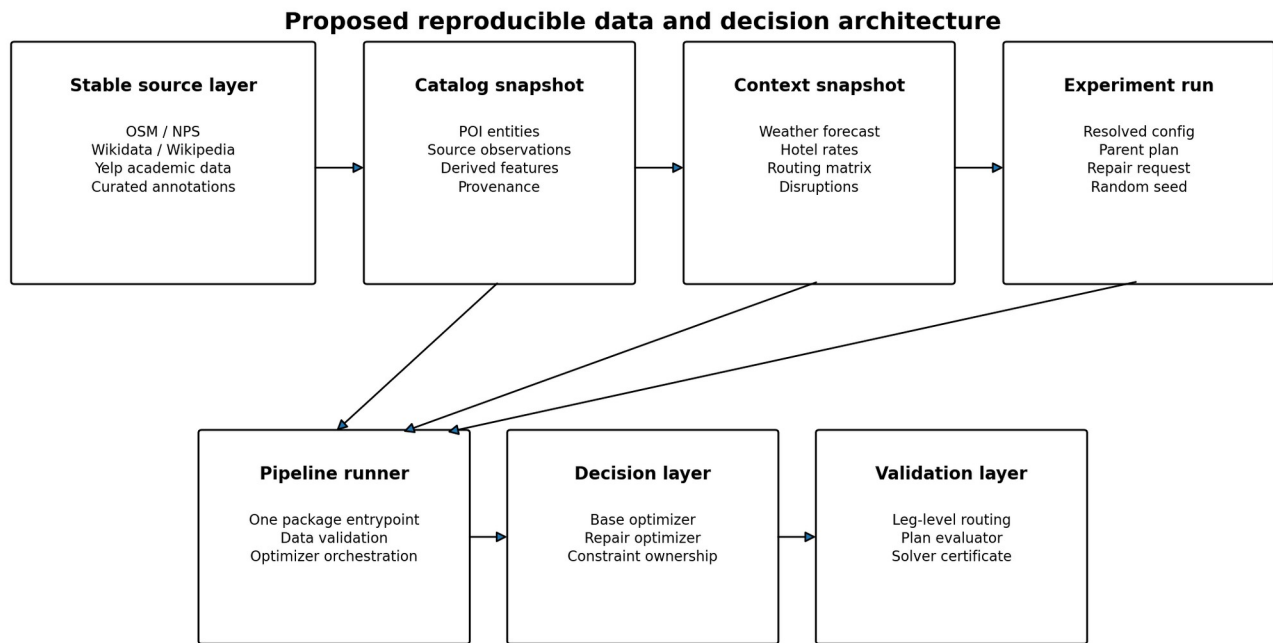


Figure 1. Stable catalog, dynamic context, and run-specific artifacts.

10.1 Stable catalog snapshot

The catalog snapshot represents relatively stable travel entities and research annotations. It should be reproducible, versioned, and safe to use without live APIs.

- POI and hotel entity identities, coordinates, and external identifiers.
- Source-level observations from OSM, NPS, Wikidata, Wikipedia, and optional Yelp extracts.
- Versioned curated annotations such as must-go status, category mappings, duration priors, and ownership examples.
- Derived semantic and interest features with explicit feature provenance.
- Entity-resolution decisions and matching confidence.

10.2 Time-sensitive context snapshot

Context snapshots preserve information that changes with time or query conditions. They are not part of the immutable base catalog.

- Weather observations, forecasts, and scenario forecasts.
- Hotel availability, rate, refundability, meal-plan, and occupancy responses.
- Routing matrices, selected road geometries, and source timestamps.
- Road closures, attraction closures, and disruption events.
- The API environment used, such as sandbox versus production.

10.3 Run artifacts

```
runs/<run_id>/
├── resolved_config.json
├── catalog_manifest.json
├── context_manifest.json
├── parent_plan.json
├── repair_request.json
├── repaired_plan.json
├── solver_certificate.json
├── plan_diff.json
├── route_leg_audit.parquet
├── evaluation.json
└── dashboard/
```

11. Problem 3: Simulated Demand and Congestion

The current fallback constructs daily review density by distributing total Yelp review counts across sampled dates and adjusting the value using manually selected weekend and weather multipliers. This is an engineered scenario, not an observed queue or arrival process.

11.1 Revised terminology

Current term	Recommended term	Reason
review_density	demand_proxy_value	The quantity is a proxy rather than direct density
review_density_source	demand_proxy_source	The source may be a simulation recipe
predicted_waiting_time	simulated_waiting_time	Avoids implying a trained model with observed targets
congestion_prediction	demand_scenario_estimate	Suitable for controlled disruption experiments

11.2 Required metadata

```
{
  "demand_proxy_value": 4.7,
  "demand_proxy_source": "business_review_count_weather_proxy",
  "is_observed": false,
  "is_simulated": true,
  "simulation_method": "review_volume_weekend_weather_v1",
  "simulation_version": "1.0",
  "ground_truth_available": false
}
```

For the first paper, the proxy should be used only to generate controlled normal, moderate-delay, and severe-delay scenarios. It should not be used to claim real-world congestion prediction accuracy.

12. Problem 4: Multi-Source POI Coverage and Fusion

The geographic coverage problem should be addressed through source-specific caching, entity resolution, and calibrated feature fusion. The goal is not to create one silently merged table; the goal is to retain every source observation and generate a transparent optimizer view.

12.1 Source responsibilities

Source	Primary role	Snapshot policy
OSM/Overpass	Base POI universe, coordinates, types, parks, trails, viewpoints, hotels	Stable catalog; cache raw queries
NPS	Authoritative national-park identity, activities, fees, hours, and descriptions	Stable catalog with retrieval timestamp
Wikidata	Structured identifiers and semantic links	Stable enrichment
Wikipedia	Description, image, and informational link	Stable enrichment with attribution
Yelp academic data	Optional rating and review-volume signal	Private external input; derived extracts only when license permits
Curated annotations	Scenario anchors, must-go labels, category corrections, duration priors	Versioned source-controlled files
Google Places	Optional rating, rating count, business status, address, website, and photos	Context or optional enrichment; not a required benchmark dependency

12.2 Cache layout and manifests

```

data_external/cache/
├── osm/<query_hash>.json
├── nps/<query_hash>.json
├── wikidata/<query_hash>.json
├── wikipedia/<query_hash>.json
├── google_places/<query_hash>.json
├── liteapi/<query_hash>.json
└── manifests/<query_hash>.manifest.json

{
  "source": "osm_overpass",
  "query_hash": "abc123",
  "query_parameters": {"city": "Monterey", "radius_m": 18000},
  "fetched_at": "2026-06-28T16:30:00Z",
  "http_status": 200,
  "schema_version": "1.0",
  "payload_sha256": "...",
  "redistribution_allowed": true
}

```

12.3 Entity resolution

OSM, Yelp, Wikidata, Google Places, and NPS records should map to a stable internal identifier without losing their original source records.

1. Match exact external IDs when an OSM record already contains a Wikidata or Wikipedia identifier.
2. Apply normalized name, geographic distance, category compatibility, address, and website-domain matching.
8. Store match score, match method, geographic distance, and manual-review status.
9. Require human review for ambiguous high-impact entities such as hotels, national parks, and locked itinerary anchors.
10. Never overwrite a source observation during entity fusion.

12.4 Missing-source-aware fusion

A missing Yelp signal must not be represented as a Yelp score of zero. Use a masked weighted fusion:

$$u_i = (\sum_s w_s m_{is} z_{is}) / (\sum_s w_s m_{is})$$

$$c_i = (\sum_s w_s m_{is}) / (\sum_s w_s)$$

Here, u_i is the fused utility signal and c_i is source coverage. They must be stored and reported separately.

12.5 Source ablation study

Condition	Sources	Purpose
A	OSM + NPS	Open authoritative baseline
B	OSM + NPS + Wikidata/Wikipedia	Effect of semantic enrichment
C	OSM + NPS + Yelp	Effect of popularity data where available
D	Full multi-source fusion	Complete operational system

Compare selected-stop overlap, feasibility, route utility, weighted edit cost, locked-item preservation, and source coverage across conditions. This turns incomplete geographic coverage into a robustness analysis rather than a hidden limitation.

13. Problem 5: Source Coverage and Uncertainty

Rename the current `data\_confidence` field to `source\_coverage\_score`, but also decompose it into interpretable dimensions.

```
identity_coverage
location_coverage
semantic_coverage
popularity_coverage
temporal_coverage
routing_coverage
source_coverage_score
```

Quantity	Meaning	Must not be confused with
source_coverage_score	How much source evidence is available for required fields	Probability that a record is correct
entity_match_confidence	Strength of cross-source identity matching	Popularity or utility
annotation_uncertainty	Reviewer disagreement or uncertain curated labels	Missing API coverage
model_uncertainty	Uncertainty generated by a statistical model	One minus source coverage

14. Problem 6: Notebook and Package Integration

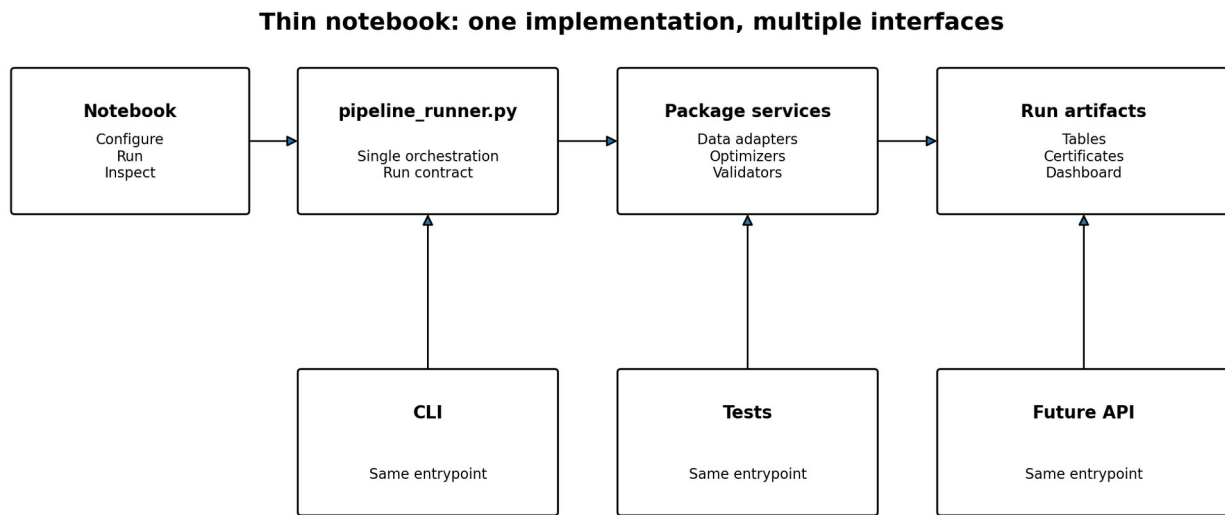


Figure 2. The notebook, CLI, tests, and future API share one package endpoint.

The notebook currently imports reusable modules but still implements substantial data acquisition and feature logic directly in cells. This creates two partially independent implementations. The revised notebook should only configure, execute, inspect, and visualize the package pipeline.

14.1 New package endpoint

```
# src/itinerary_system/pipeline_runner.py

def run_research_pipeline(
    *,
    config_path: Path,
    catalog_snapshot_id: str,
    context_snapshot_id: str | None = None,
    refresh_policy: str = "never",
    run_id: str | None = None,
) -> PipelineRun:
    ...

@dataclass
class PipelineRun:
    run_id: str
    catalog_bundle: DatasetBundle
    context_bundle: ContextBundle
    parent_plan: PlanArtifact
    repaired_plans: list[PlanArtifact]
    evaluation: EvaluationBundle
    output_dir: Path
```

14.2 Notebook cell contract

Cell group	Responsibility	Forbidden responsibility
Environment	Import installed package and display versions	Mutating sys.path or repeatedly reloading core modules

Cell group	Responsibility	Forbidden responsibility
Configuration	Select YAML, snapshot IDs, and run mode	Defining duplicate defaults
Data	Load or explicitly refresh a dataset bundle	Embedding acquisition and entity-resolution logic
Validation	Display schema, coverage, and blocking errors	Silently coercing invalid records
Execution	Call <code>`run_research_pipeline()`</code>	Calling individual optimizers with ad hoc DataFrames
Inspection	Display route, diff, audit, and solver evidence	Recomputing metrics separately
Export	Call the package dashboard exporter	Building a second dashboard data model

14.3 Snapshot and run identifiers

```
catalog_snapshot_id = "california_v1"
context_snapshot_id = "context_2026_06_28T160000Z"
run_id = "repair_weather_001_seed42"
```

Every optimizer result, evaluation row, dashboard, and solver certificate must reference all three identifiers. This guarantees that a result can be regenerated from the same stable entities and the same time-sensitive operating context.

14.4 Weather modernization

```
today = datetime.now(UTC).date()

if trip_date < today:
    weather_mode = "historical_archive"
elif trip_date <= supported_forecast_end:
    weather_mode = "forecast"
else:
    weather_mode = "seasonal_scenario"
```

Field	Purpose
weather_snapshot_id	Links the weather record to the experiment context
location_id	Stable city, park, or coordinate reference
valid_time	When the weather applies
forecast_issue_time	When the forecast was produced
retrieved_at	When the project fetched it
weather_mode	Historical, forecast, or scenario
is_observed / is_forecast / is_scenario	Prevents category confusion

15. Problem 7: Hotel Identity, Rates, and Availability

The recommended hotel architecture is OSM for stable identity, LiteAPI for date-specific rate and availability observations, and Google Places only for optional metadata enrichment. LiteAPI documents real-time rates and availability, room options, cancellation policies, meal plans, and multiple search methods including hotel IDs, city,

coordinates, Place ID, and IATA code. Its documentation also distinguishes free sandbox access from production booking credentials [1-2].

Credential action

The LiteAPI sandbox credential shared during discussion must be rotated. The report intentionally excludes all credential values. Secrets must be loaded from environment variables or a local ignored `.env` file and must never be serialized into resolved configuration artifacts.

15.1 Stable hotel entity table

```
hotel_id
canonical_name
latitude
longitude
address
osm_id
liteapi_hotel_id
google_place_id
entity_match_confidence
entity_match_method
match_reviewed
```

15.2 Time-specific rate observation

```
hotel_id
rate_snapshot_id
provider
provider_hotel_id
environment
check_in
check_out
occupancy
currency
nightly_price
total_price
taxes
refundable
meal_plan
availability_confirmed
queried_at
expires_at
raw_payload_hash
```

15.3 Entity matching policy

1. Reuse an existing provider-ID link whenever available.
2. Filter candidate matches by geographic distance.
8. **Compare normalized hotel names and address components.**
9. **Use website/domain agreement as supporting evidence.**
10. **Store match score and method.**
11. **Require manual review for ambiguous matches; do not merge solely by name.**

15.4 Proposed cache policy

Data type	Initial project policy	Required metadata
OSM hotel identity	30-90 days	OSM data timestamp, query, payload hash

Data type	Initial project policy	Required metadata
LiteAPI hotel metadata	7-30 days	Environment, queried_at, provider hotel ID
LiteAPI rate/availability	6-24 hours	Dates, occupancy, currency, queried_at, sandbox/production
Google Places metadata	7-30 days	Field mask, queried_at, billing SKU context
Hotel entity links	Until reviewed or invalidated	Match score, method, reviewer state

These are internal research cache policies, not claims that a rate remains bookable for the full period. Every displayed rate must show its query time and provider environment.

15.5 Configuration without secrets

```
hotels:
  identity_source: osm
  live_rate_provider: liteapi
  liteapi_environment: sandbox
  liteapi_key_env: LITEAPI_SANDBOX_KEY
  optional_metadata_provider: google_places
  google_key_env: GOOGLE_MAPS_API_KEY
# Local .env only - never commit
LITEAPI_SANDBOX_KEY=...
GOOGLE_MAPS_API_KEY=...
```

16. Problem 8: Routing Provenance and Road Validation

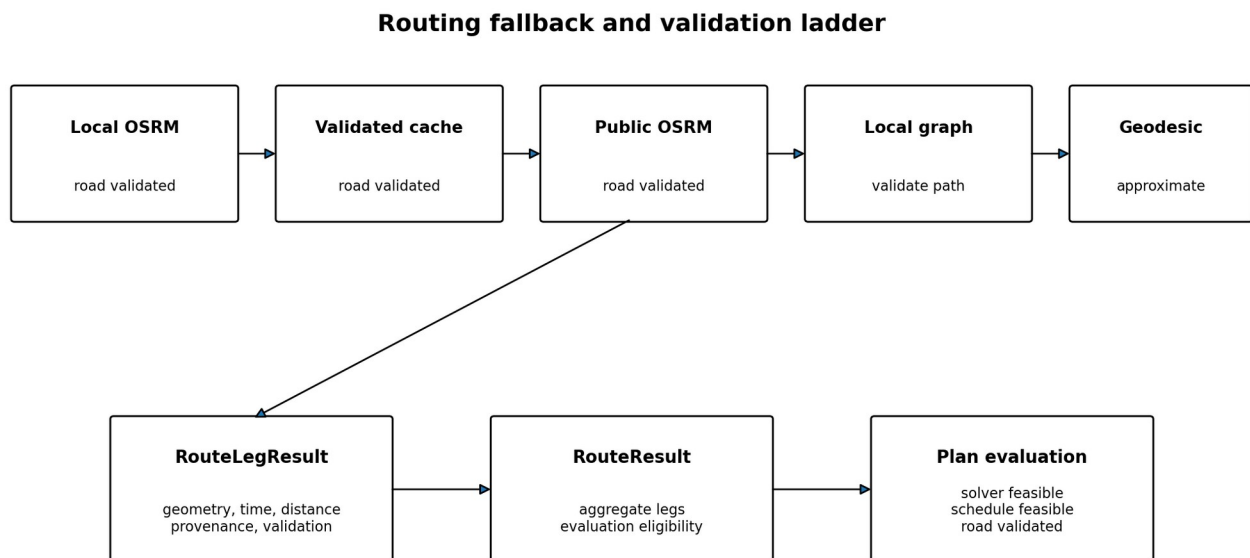


Figure 3. Routing fallback ladder and leg-level validation.

The current helper can return straight waypoint lines when OSRM is unavailable. That behavior is acceptable for debugging, but it must be represented as an approximate fallback and excluded from road-valid evaluation. The optimizer, dashboard, and evaluator must consume the same route-leg records.

16.1 Structured routing objects

```
@dataclass(frozen=True)
class RouteLegResult:
    origin_id: str
    destination_id: str
    geometry: list[tuple[float, float]]
    distance_m: float | None
    duration_s: float | None
    routing_status: str
    provider: str
    routing_profile: str
    geometry_source: str
    distance_source: str
    duration_source: str
    road_validated: bool
    fallback_used: bool
    fallback_reason: str | None
    query_hash: str
    retrieved_at: datetime | None
    cache_created_at: datetime | None
    snap_distance_origin_m: float | None
    snap_distance_destination_m: float | None

@dataclass(frozen=True)
class RouteResult:
    route_id: str
    legs: tuple[RouteLegResult, ...]
    total_distance_m: float | None
    total_duration_s: float | None
    road_validated: bool
    evaluation_eligible: bool
```

16.2 Feasibility states

```
road_validated = all(leg.road_validated for leg in legs)

evaluation_eligible = (
    solver_feasible
    and schedule_feasible
    and road_validated
    and dataset_snapshot_valid
)
```

State	Meaning
solver_feasible	The mathematical constraints are satisfied under the supplied coefficients
schedule_feasible	The postprocessed day schedule satisfies time windows and duration calculations
road_validated	Every required movement uses a road-network route or validated road cache
evaluation_eligible	The plan can enter the paper's final road-valid comparison

16.3 OSRM service roles

OSRM exposes separate Route and Table services. The Table service should build travel-time and distance matrices for the optimizer, while the Route service should retrieve selected geometry and leg details. OSRM also exposes fallback-speed cells; those cells must be labeled approximate rather than road validated [3].

Service	Project use	Output policy
OSRM Table	Candidate travel-time and distance matrices	Persist query hash, provider version, fallback cells, and timestamps
OSRM Route	Selected route geometry and detailed legs	Persist geometry, duration, distance, snapping evidence, and validation
OSRM Nearest	Point-to-network snapping validation	Store snap distance and reject excessive snapping

16.4 Routing-source policy

Source	Display	Optimize	Final evaluation
Local OSRM	Yes	Yes	Yes
Validated OSRM cache	Yes	Yes	Yes
Public OSRM	Yes	Yes	Yes, with preserved response metadata
Validated OSMnx road path	Yes	Yes	Yes
OSRM fallback-speed cell	Yes	Approximate mode	No
Straight/geodesic fallback	Yes	Approximate mode	No

16.5 Precomputation scope

1. Build a complete city and gateway matrix.
2. Build hotel-to-city-anchor matrices.
8. Build k-nearest local POI connections instead of all POI pairs.
9. Retrieve full road geometry only for candidate routes entering optimization or final selected routes.
10. Freeze the benchmark routing cache before running comparative experiments.

16.6 Future scenic-road extension

OSMnx supports shortest-path and k-shortest-path analysis over a road-network multigraph, which makes it suitable for a later continuous scenic-routing contribution [4]. A future edge objective can combine travel time with highway, weather, scenic, coastal, and nature terms:

$$c_e = t_e + \lambda_h H_e + \lambda_r R_e - \lambda_s S_e - \lambda_c C_e - \lambda_n N_e$$

This local-graph extension should not delay the first repair paper. The initial benchmark can use validated OSRM routes and treat scenic-corridor preservation as a user-owned route constraint.

17. Unified Data Contracts

17.1 Catalog tables

Table	Key contents	Update rate
poi_entities	Stable IDs, canonical names, coordinates, entity types, external IDs	Infrequent
poi_observations	One source record per POI and source	Per source snapshot
poi_features	Derived semantic, interest, duration, and utility features	Per feature version
feature_provenance	Input source IDs, transformations, observed/curated/imputed/simulated flags	Every feature build
hotel_entities	Stable hotel identities and provider links	Infrequent
curated_annotations	Must-go labels, category corrections, priors, ownership examples	Versioned manual update

17.2 Context tables

Table	Key contents
weather_snapshots	Valid time, issue time, values, weather mode, source
hotel_rate_snapshots	Dates, occupancy, price, availability, policy, environment, query time
route_legs	Origin, destination, geometry, time, distance, source, validation
disruption_snapshots	Closure, weather, hotel, fatigue, or road event used in repair

17.3 Decision and benchmark tables

Table	Key contents
plan_artifacts	Plan lineage, dataset IDs, solver run, certification
repair_instances	Parent plan, disruption, ownership profile, allowed neighborhood
plan_diffs	Added, deleted, moved, reordered, hotel, and road changes
evaluation_results	Feasibility, edit cost, utility, route validation, explanation correctness

18. Recommended Repository Structure

```

data/
├── README.md
├── registry/sources.yaml
├── schemas/
├── curated/
├── snapshots/california_v1/
└── benchmark/

data_external/          # ignored
├── yelp/
└── cache/

src/itinerary_system/
├── data/
│   ├── registry.py
│   ├── schemas.py
│   ├── entity_resolution.py
│   ├── provenance.py
│   ├── snapshot.py
│   └── adapters/
├── hotels/
│   ├── liteapi_adapter.py
│   ├── google_places_adapter.py
│   └── matching.py
├── routing/
│   ├── models.py
│   ├── osrm_adapter.py
│   ├── osmnx_adapter.py
│   ├── matrix.py
│   ├── cache.py
│   └── validation.py
├── pipeline_runner.py
├── repair_optimizer.py
├── plan_evaluator.py
└── explanation_engine.py

```

19. File-Level Implementation Plan

File or module	Change	Acceptance criterion
`config.py`	Add snapshot IDs, refresh policies, hotel provider settings, routing policies; redact secret values	Resolved config contains environment-variable names but no credential values
`data_enrichment.py`	Split adapters and feature assembly; preserve source observations	Canonical optimizer view can be rebuilt from catalog tables
`nature_catalog.py`	Move curated seeds and keyword mappings to versioned data files	No high-impact prior is hidden in Python constants
`utility_model.py`	Implement masked source fusion and source coverage components	Missing Yelp does not contribute a zero score
`pipeline_runner.py`	Create the single orchestration entrypoint	Notebook, CLI, and tests produce matching run manifests
`hotels/liteapi_adapter.py`	Add rate search, sandbox labeling, caching, and redaction	No raw credential or full response leaks into dashboard

File or module	Change	Acceptance criterion
`routing/models.py`	Add RouteLegResult and RouteResult	Every route value has a source and validation flag
`routing/osrm_adapter.py`	Separate Table, Route, and Nearest calls	Optimizer matrix and map geometry use linked route records
`plan_evaluator.py`	Require road validation for final comparison	Approximate routes remain visible but are evaluation-ineligible
Production notebook	Replace duplicated logic with package calls	A clean execution can reproduce all tables and dashboards
`data/snapshot.py` and `data/manifest.py`	Load exact immutable snapshots, verify hashes, and build/promote new versions	A clean clone executes without reading `results/outputs` or private Yelp files
`data/schemas.py` and `data/provenance.py`	Define evidence kinds, layered records, and feature-lineage requirements	Every optimizer-facing non-key field has reconstructable provenance
`data/optimizer_view.py`	Materialize the validated wide table consumed by optimization	The current optimizer schema is reproduced without treating the wide table as source truth

20. Staged Implementation Roadmap

Stage	Tasks	Deliverable
1. Safety and terminology	Rotate exposed credential; rename coverage fields; label simulation and routing fallbacks	No secret leakage; honest current artifacts
2. Data foundation	Source registry, schemas, entity IDs, provenance, fallback snapshot, masked fusion	`california_v1` catalog snapshot
3. Notebook integration	Pipeline runner, run manifests, common CLI/notebook/test entrypoint	One reproducible execution path
4. Weather and context	Time-aware weather adapter and context snapshot builder	Frozen weather scenarios for repair experiments
5. Hotel integration	OSM entities, LiteAPI matching and rates, optional Google metadata	Timestamped hotel-rate context
6. Routing consistency	Leg models, OSRM matrix/route separation, road-validation audit	Road-valid benchmark cache
7. Repair benchmark	Parent plans, disruptions, ownership profiles, baselines, metrics	Pilot 60-case benchmark, then main benchmark
8. Scenic extension	Local graph and custom edge features after first paper is stable	Second-paper technical direction

21. Evaluation Plan

21.1 Data-quality evaluation

- Entity-resolution precision on a manually reviewed sample.

- Coverage by city, source, POI category, and required feature.
- Missingness patterns before and after source fusion.
- Agreement between source-specific category and popularity indicators.
- Fraction of selected itinerary stops with complete provenance.
- Fraction of route legs that are road validated.

21.2 Repair-system evaluation

Metric	Definition
Hard-constraint feasibility	All user locks, bookings, time windows, and safety constraints are satisfied
Weighted edit cost	Deletes, additions, day moves, reorderings, hotel changes, and road changes
Unaffected-day preservation	Share of non-disrupted itinerary days unchanged
Locked-item preservation	Share of user-owned locked items retained
Utility and weather exposure	Decision quality after repair
Road-valid eligibility	Share of outputs entering final evaluation
Explanation correctness	Whether every explanation statement is supported by structured evidence
Runtime and solver certificate	Optimization status, gap, fallback behavior, and certificate validity

21.3 Required ablations

- Source ablation: OSM+NPS versus semantic enrichment versus Yelp versus full fusion.
- Context ablation: fixed priors versus time-specific hotel and weather context.
- Routing ablation: geodesic approximate mode versus validated OSRM cache.
- Repair ablation: full regeneration versus full reoptimization versus progressive local repair.
- Ownership ablation: no ownership labels versus visible ownership and graded edit strength.

22. Required Automated Tests

```

test_clean_clone_has_valid_catalog_snapshot
test_snapshot_manifest_hashes_match
test_all_optimizer_features_have_provenance
test_missing_yelp_is_not_zero_yelp
test_no_duplicate_stable_entity_ids
test_ambiguous_hotel_matches_require_review
test_resolved_config_redacts_secrets
test_sandbox_hotel_rates_are_labeled
test_weather_snapshot_contains_valid_and_issue_times
test_route_matrix_and_geometry_share_route_records
test_osrm_fallback_cells_are_not_road_validated
test_straight_fallback_is_evaluation_ineligible
test_notebook_and_cli_share_pipeline_runner
test_no_cross_split_parent_plan_leakage
test_generated_outputs_are_never_dataset_inputs
test_missing_snapshot_fails_with_actionable_error
test_every_optimizer_feature_has_evidence_kind
test_decision_columns_absent_from_catalog_tables
test_optimizer_view_rebuild_is_deterministic

```

23. Security, Licensing, and Reproducibility

- Rotate any credential shared in chat, issue trackers, notebooks, or local logs.
- Use `.env` only for local development and keep it ignored.
- Redact secrets from exceptions, resolved configs, experiment manifests, and dashboard payloads.
- Do not redistribute commercial API payloads unless provider terms permit it.
- Preserve OSM attribution and ODbL obligations for shared derived databases [5].
- Preserve provider attribution and licensing notes for Open-Meteo and other data sources.
- Treat Yelp academic data as a local external dependency and document the acquisition process instead of committing raw files.
- Record retrieval timestamps, query parameters, payload hashes, code commit, schema version, and transformation version.

Part III

Implementation, Benchmark, and Preliminary Paper Design

24. Repository-Specific Implementation Architecture

The implementation should extend the Phase 0 foundation instead of introducing a parallel architecture. Existing snapshot, routing-model, artifact, and validator contracts become the base layer. New plan/repair modules should be focused, typed, and usable from the notebook, CLI, tests, and future dashboard service.

24.1 Target package structure

```
src/itinerary_system/
├── data/
│   ├── schemas.py          # existing DatasetBundle, extend carefully
│   ├── snapshot.py         # stable/context loaders and validation
│   ├── provenance.py       # field-level evidence records
│   └── optimizer_view.py    # read-only joined solver inputs
├── lodging/
│   ├── models.py           # category taxonomy, entities, rate snapshots
│   ├── provider.py         # provider protocol + activation result
│   ├── osm_adapter.py
│   ├── geoapify_adapter.py
│   ├── liteapi_adapter.py
│   ├── google_places_adapter.py
│   ├── booking_adapter.py  # dormant without partner credentials
│   └── expedia_adapter.py  # dormant without partner credentials
├── routing/
│   ├── models.py           # existing RouteLegResult/RouteResult
│   ├── cache.py            # existing validated cache
│   ├── osrm_client.py
│   ├── matrix.py
│   ├── validation.py
│   └── docker/             # compose, profiles, documented commands
├── plans/
│   ├── models.py           # PlanArtifact v2, OwnedConstraint, EditRequest
│   ├── repository.py       # immutable read/write + content hashes
│   └── diff.py
├── repair/
│   ├── neighborhood.py
│   ├── master_model.py
│   ├── day_route_solver.py
│   ├── lexicographic.py
│   ├── progressive.py
│   └── relaxation.py
├── evaluation/
│   ├── plan_evaluator.py
│   ├── certificate.py
│   └── metrics.py
├── explanation/
│   ├── evidence.py
│   ├── counterfactual.py
│   └── verbalizer.py
├── pipeline_runner.py
└── experiment_runner.py    # benchmark only, not general orchestration
```

24.2 Two-stage notebook migration

6. Stage A: preserve the documented notebook command, but replace acquisition, simulation, merge, optimization, and export cells with calls to `load_dataset_bundle()` and `run_research_pipeline()`. The notebook only resolves config, runs, displays, and exports.

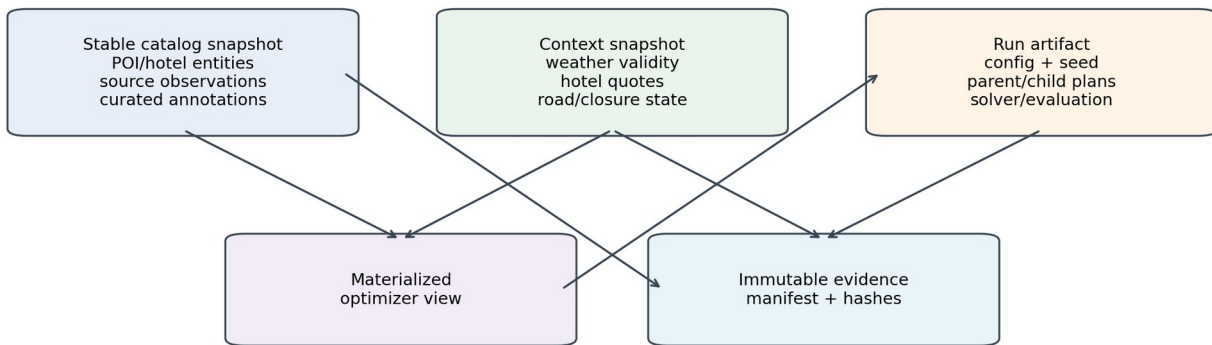
7. Stage B: make a CLI the authoritative execution path. The notebook becomes a visualization and analysis client that loads an immutable run directory. The dashboard service later calls the same pipeline runner rather than executing notebook state.

```
run = run_research_pipeline(
    config_path=Path('configs/nature_trip_config.yaml'),
    catalog_snapshot_id='california_v1',
    context_snapshot_id='context_2026_07_road_validated_v1',
    parent_plan_id='plan_california_repair_v1',
    repair_request_id='repair_weather_001',
    refresh_policy='never',
)
```

24.3 Code readability and Codex contract

- Python 3.12, complete public type hints, frozen dataclasses for immutable research artifacts, and enums for closed vocabularies.
- No API calls inside model builders; providers and routing clients are injected dependencies.
- No notebook-only business logic and no hidden module-level mutable state.
- Short pure functions for transformations; explicit structured exceptions for unavailable providers, invalid snapshots, and certification failures.
- Deterministic seeds, stable hashes, explicit schema versions, and no secret values in logs or resolved configs.
- Parent plans are never mutated. Every accepted modification is a new child with a content hash and parent identifier.
- Compatibility wrappers remain temporarily but include deprecation comments and tests; unreachable legacy code is removed after equivalence checks.

Mandatory catalog-context-run separation



Time-sensitive quotes and forecasts never overwrite stable entity records.

Figure 6. Catalog, context, and run artifacts as the authoritative pipeline contract.

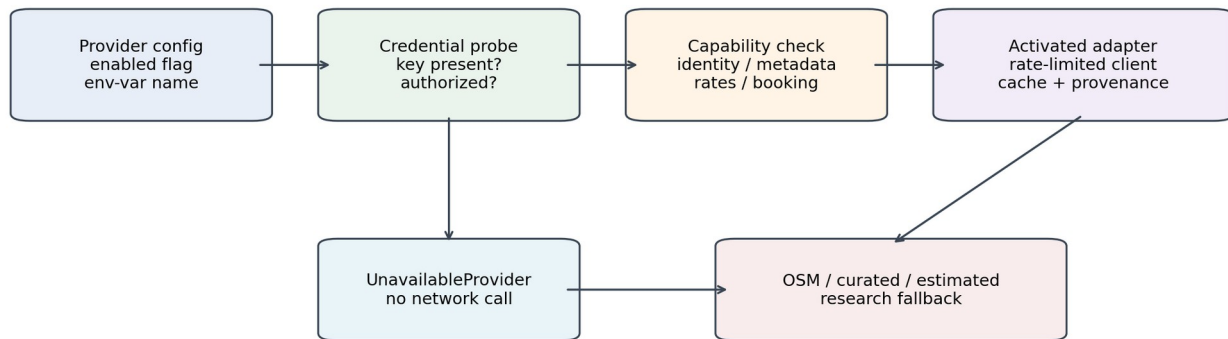
24.4 Lodging taxonomy and credential-gated providers

Canonical category	Examples / mapping notes
hotel	hotel, boutique hotel, aparthotel when hotel-operated
motel	roadside motel and motor inn
hostel	hostel and youth hostel
guest_house	guest house and lodge when not a full hotel
bed_and_breakfast	B&B and inn with hosted breakfast

Canonical category	Examples / mapping notes
resort	resort and resort hotel
extended_stay	extended-stay hotel or serviced residence
apartment	serviced apartment when inventory behaves as lodging
vacation_rental	whole-home short-term rental; provider-neutral
private_room	private guest room or homestay room
campground	campground and camp site
camping_cabin	camping cabin, glamping cabin, wilderness hut when bookable
rv_park	RV park / caravan site
other	unmapped accommodation requiring review

Geoapify is a useful optional enrichment provider because its free plan currently offers 3,000 credits per day, can be started without a credit card, and permits storage of created data subject to its terms and attribution. LiteAPI remains the optional date-specific rate and availability provider; every account can obtain a sandbox key, but sandbox responses must be labeled as non-production and production booking access requires a payment method. Google Places is optional for ratings, business status, website, address verification, and photos. Booking.com and Expedia Rapid adapters may be implemented behind the provider protocol but remain inactive without the required commercial partnership. An Airbnb adapter is out of scope unless official partner access exists; undocumented scraping or APIs are prohibited.

Commercial API activation gate



Adapters are implemented before keys are supplied; secrets never enter resolved configs or artifacts.

Figure 7. Provider adapters activate only after explicit credential and capability checks.

```

class LodgingProvider(Protocol):
    provider_id: str

    def probe_credentials(self) -> ProviderActivation:
        ...

    def search_entities(self, query: LodgingSearchQuery) -> tuple[LodgingObservation, ...]:
        ...

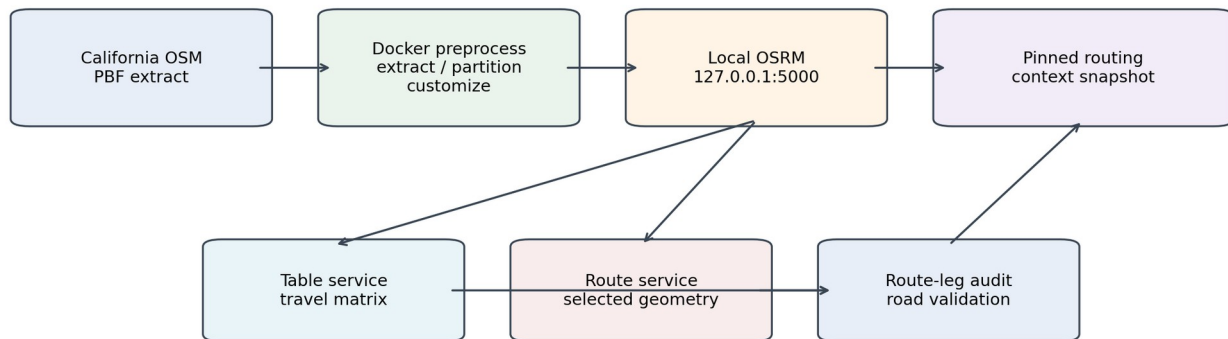
    def fetch_rates(self, query: RateQuery) -> tuple[HotelRateSnapshot, ...]:
        ...
  
```

No key or invalid key -> provider is unavailable; no network request in normal runs.

24.5 Local OSRM Docker is a core infrastructure milestone

The Phase 0 scripts already default to a local OSRM endpoint and intentionally block the public demo service unless explicitly allowed. The next step is to commit a reproducible Docker workflow for a pinned California extract and OSRM image. The optimizer uses Table responses; selected routes use Route responses; final artifacts store the endpoint policy, map-extract checksum, profile, retrieval time, query hash, and route-leg validation state.

Reproducible local OSRM deployment



Public endpoints are optional and explicit; final comparisons prefer pinned local routing evidence.

Figure 8. Local OSRM deployment and routing-context freeze.

```

docker compose -f docker/osrm/docker-compose.yml run --rm osrm-preprocess

docker compose -f docker/osrm/docker-compose.yml up -d osrm-router

python scripts/check_route_source.py --output-dir results/outputs --osrm-base-url http://127.0.0.1:5000 --probe
  
```

25. Six-Disruption Repair Benchmark

The benchmark unit is a repair instance, not a POI row. Each instance binds one immutable parent plan, one disruption, one ownership profile, one confirmed edit request, one catalog snapshot, and one context snapshot. Development, validation, and test splits must separate complete parent-plan/disruption families to prevent near-duplicate leakage.

Disruption	Model change	Expected repair behavior
Outdoor weather deterioration	F_id(omega)=0 or increased weather risk for outdoor POIs on affected day.	Move, replace with indoor option, or expand radius while preserving locks.
Road closure	Remove or penalize affected road arcs; mark preferred corridor unavailable.	Reroute with explicit scenic-preference relaxation evidence.
Hotel unavailability	Set z_hd=0 for unavailable lodging/date.	Preserve other nights; suggest smallest lodging relaxation.
Attraction closure / opening conflict	Update assignment/time-window feasibility.	Replace or move the affected stop; keep unrelated days frozen.
Reduced driving tolerance / fatigue	Lower daily travel budget for selected days.	Local rescheduling, optional-stop removal, or hotel adjustment.

Disruption	Model change	Expected repair behavior
New user must-visit	Force $v_i=1$ with specified strength/day flexibility.	Find minimal additions/removals; explain trade-offs and infeasibility.

25.1 Parent plans and ownership profiles

- `plan_demo_current.json`: faithfully serialized from the current saved 7-day GitHub route for regression and migration testing.
- `plan_california_repair_v1.json`: hand-reviewed benchmark parent using Los Angeles - Santa Barbara - Big Sur/Monterey - Yosemite - San Francisco.
- Ownership profiles: flexible, mixed, booking-constrained, scenic-route-protective, and high-lock.
- Trip variants: 7-, 9-, and 12-day routes plus reverse gateway direction after the core benchmark passes.

25.2 Benchmark sizes and baselines

Stage	Suggested size	Purpose
Smoke	1 parent x 6 disruptions x 2 profiles	Test every pathway, artifact, and certificate.
Pilot	12 parents x 6 disruptions	Estimate feasibility, runtime, and metric variation.
Main	20 parents x 6 disruptions x 3 ownership profiles $\approx$ 360 cases	Computational paper evidence with held-out scenario families.

Baseline	Definition
LLM-only rewrite	Text model revises the itinerary without a formal solver certificate.
Full reoptimization	Optimizer solves the disrupted instance without parent-preservation objectives.
Current hybrid pipeline	Existing ranking/bandit candidate selection plus current small Gurobi route solver.
Weighted-sum repair	Parent-change penalties mixed into one objective.
Fixed-radius local repair	Single predetermined neighborhood, no expansion.
Proposed method	Ownership-aware sequential lexicographic repair with progressive neighborhoods and independent validation.

26. Algorithmic Evaluation and Statistical Analysis

26.1 Primary outcomes

Family	Measures
Feasibility	Hard violations, route connectivity, time-window compliance, hotel continuity, closure/weather violations.
Preservation	Weighted edit cost, locked/booked preservation, unaffected-day preservation, day moves, hotel changes, road changes, order distance.
Quality	Utility, diversity, weather exposure, road-valid travel, estimated cost, scenic preference fit, utility regret.

Family	Measures
Computation	Runtime, radius expansions, solver stages, MIP gap, fallback rate, certificate coverage.
Evidence	Artifact grounding, route-source completeness, unsupported explanation claims, counterfactual consistency.

26.2 Statistical plan

- Use paired comparisons because each method receives the same parent plan, disruption, ownership profile, catalog, and context snapshot.
- Report median and interquartile range for skewed runtime/edit metrics; report mean and confidence interval where distributions justify it.
- Use mixed-effects models or cluster-robust analysis with parent plan and disruption family as grouping factors.
- For binary feasibility/certificate outcomes, use paired proportions and generalized mixed models rather than treating each POI as independent.
- Report effect sizes and uncertainty, not only p-values. Correct families of exploratory comparisons and label preregistered primary outcomes.
- Conduct ablations for ownership, lexicographic priorities, progressive neighborhoods, road validation, source fusion, and independent verification.

26.3 Source robustness

Because Yelp coverage is geographically uneven, run the repair benchmark under OSM+NPS, OSM+NPS+Wikidata/Wikipedia, OSM+NPS+Yelp where available, and full multi-source fusion. Missing Yelp must be represented by a mask rather than a zero score. Report whether repair decisions and preservation metrics are stable across source configurations.

27. IUI-First, CHI-Compatible User-Study Protocol

The user study should evaluate the control loop, not whether participants like colorful maps. Participants receive a partially planned California trip, a disruption, and a concrete decision task. They must inspect the system interpretation, resolve one ambiguous edit, compare repairs, and decide whether to accept a child plan.

27.1 Hypotheses

Hypothesis	Prediction
H1 Constraint comprehension	Visible ownership and strength improve identification and correction of system-interpreted constraints.
H2 Preservation understanding	Plan diff plus progressive repair improves recognition of what changed and what remained protected.
H3 Calibrated reliance	Certificates, warnings, and evidence links reduce acceptance of invalid or unsupported repairs without reducing acceptance of valid ones.
H4 Perceived control	Direct edits with strength confirmation increase plan ownership and perceived control relative to opaque regeneration.
H5 Decision efficiency	Consequence previews reduce unnecessary iterations and help participants choose acceptable repairs faster.

27.2 Conditions

Condition	Interface
C1 Regeneration baseline	LLM or optimizer presents a revised itinerary without ownership/diff controls.
C2 Editable full-replan	Participant edits a stop; system regenerates the complete itinerary.
C3 Minimal repair	Ownership-aware progressive repair and before/after diff, but limited explanations.
C4 Full proposed system	Constraint inspection, edit strength, progressive repair, certificate, evidence-grounded explanation, and what-if.

27.3 Tasks and measures

- Task 1: identify an incorrectly inferred rule and correct its scope or strength.
- Task 2: respond to one weather, road, or hotel disruption while preserving specified commitments.
- Task 3: issue a direct edit whose meaning is ambiguous and choose the intended semantics.
- Task 4: compare a minimal-change and globally optimized repair and justify acceptance.
- Task 5: inspect a deliberately unsupported explanation or invalid route warning.

Measure type	Examples
Objective	Task success, rule-correction accuracy, violation detection, acceptance calibration, time, iterations, undo actions.
Mental model	Predict what will change, identify binding constraint, distinguish solver feasibility from road validation.
Experience	Perceived control, plan ownership, workload, explanation satisfaction, confidence in decision.
Behavioral logs	Edit events, strength selections, preview requests, counterfactuals, accepted plan, evidence-panel openings.
Qualitative	Think-aloud and interview themes about unwanted change, trust repair, reservation protection, and control.

27.4 Analysis and ethics

Use a within-subject or mixed design with counterbalanced scenarios, avoiding repeated exposure to near-identical plans. Predefine primary outcomes and manipulation checks. Store only de-identified event logs, hashed participant IDs, condition/task IDs, timing, actions, and ratings. Do not retain personal travel histories or raw conversation content unless the approved study protocol requires them. Treat trust as calibrated reliance: accepting correct repairs and rejecting incorrect ones, not merely reporting higher trust.

28. Preliminary Scenic-Routing Implementation

A preliminary scenic component is appropriate when it remains subordinate to the repair contribution. In Paper 1, Highway 1 or another corridor is an owned route preference. The master chooses among a small set of road-valid route options and charges a road-change cost when the child departs from the parent corridor. This requires no new perception model.

$$\text{ScenicFit}(q) = \sum_{e \in E} s_{e,q_e}, \quad \text{with } s_{e,q_e} \text{ from curated corridor or route-option metadata}$$

The implementation can define a `ScenicRouteFeatureProvider` interface and a `route_option.scenic_score` field. The first version may use curated Highway 1 membership, coast proximity, viewpoints, and protected-area exposure. It must label those values as curated or derived. OSMnx edge-level routing becomes a later module only after the repair benchmark is stable.

$$c_e = \text{travel_time_e} + \lambda_h \text{highway_penalty_e} + \lambda_r \text{risk_e} - \lambda_s \text{scenic_e} - \lambda_c \text{coastal_e} - \lambda_n \text{nature_e}$$

This generalized edge cost is the future extension. It should not delay local OSRM routing or the first user-owned repair prototype.

29. Dependency-Ordered Development Roadmap

The roadmap deliberately avoids fixed dates. Each gate produces artifacts that can support the next venue package. The TRB-oriented gate prioritizes truthful route evidence and computational repair. The IUI gate adds the mixed-initiative prototype and controlled study. The CHI gate requires stronger interaction polish, qualitative depth, and design implications.

Gate	Dependencies	Exit evidence
G0 Repository truth	Current main, tests, clean snapshot, route audit.	All current artifacts reproducible; dead code and stale paths documented.
G1 Canonical plans and diff	PlanArtifact v2, repository, plan_diff, evaluator.	Current route serialized as parent; every mutation yields child and diff.
G2 Road-valid context	Local OSRM Docker, Table/Route client, complete benchmark cache.	Strict Phase 0 comparison gate passes.
G3 Single-day repair	OwnedConstraint, EditRequest, day-route subproblem, sequential objectives.	Weather/closure repair preserves locks and certifies final route.
G4 Progressive master	Repair master, neighborhood expansion, hotel and corridor changes.	All six disruptions execute with typed change metrics.
G5 Benchmark package	Baselines, splits, ablations, statistical scripts.	Transportation-ready computational evidence and limitations.
G6 Interactive prototype	Pipeline runner, service endpoint, map/timeline diff, edit strength, consequence preview.	IUI-ready working system.
G7 Explanation and study	Counterfactual evidence, verbalizer, event logging, pilot study.	IUI submission package; CHI feasibility decision.
G8 CHI-strength extension	Study refinement, qualitative analysis, interaction polish.	CHI-compatible paper with design implications.

30. Preliminary Paper and System-Design Outline

30.1 IUI-first paper structure

8. Introduction: AI regeneration destroys traveler-owned work; contributions and research questions.
9. Related work: itinerary generation/modification, disruption repair, mixed-initiative optimization, explainable optimization, and plan stability.
10. Design rationale: ownership, ambiguous edit semantics, local repair, consequence previews, and independent evidence.
11. System: data/context/run layers, interaction workflow, repair master, day-route subproblems, routing service, evaluator, explanation engine.

12. Formulation: ownership weights, typed change variables, sequential objectives, progressive neighborhoods, certificates.
13. Computational evaluation: six disruptions, baselines, ablations, preservation, quality, runtime, and route validity.
14. User study: conditions, tasks, comprehension, control, calibrated reliance, and qualitative findings.
15. Discussion: division of labor among traveler, LLM, optimizer, routing provider, and verifier; generalization and limitations.
16. Conclusion: repair rather than regenerate.

30.2 Transportation-oriented preliminary package

The first transportation package can omit claims about user outcomes and focus on the repair algorithm, road-validated context, disruption benchmark, stability-quality trade-off, hotel and corridor constraints, and certification. The interface may appear as a decision-support demonstrator. This is a narrower and more achievable package when the human study is not yet ready.

30.3 CHI-compatible extension

The CHI extension should foreground plan ownership, agency, consequence preview, trust calibration, and qualitative accounts of when travelers prefer stability over global optimality. It requires more than adding questionnaires to the UI paper; the discussion must connect observed behaviors to design implications for AI-assisted planning and mixed-initiative systems.

31. Claims, Risks, and Final Recommendation

31.1 Safe claims after implementation

- The system represents traveler-owned commitments explicitly and optimizes typed plan changes relative to an immutable parent plan.
- Progressive repair provides a measurable stability-quality trade-off and stops at the smallest certified feasible neighborhood.
- Every final comparison plan is grounded in fixed catalog/context snapshots and road-validated route records.
- Explanations are generated from plan diffs, active constraints, independent evaluation, and counterfactual runs rather than free-form model inference.
- The study evaluates comprehension, control, and calibrated reliance rather than assuming that explanations create trust.

31.2 Claims to avoid

- Do not claim the first disruption-aware itinerary repair system.
- Do not describe geodesic/straight-line routes as road validated or solver certified.
- Do not call source coverage calibrated uncertainty.
- Do not call hotel priors live prices or sandbox responses real availability.
- Do not call a vacation rental “Airbnb” without provider evidence.
- Do not describe the bandit as online personalization without real sequential user rewards.
- Do not claim scenic-routing novelty from corridor proximity alone.

31.3 Final recommendation

Next coherent milestone

Freeze `plan_demo_current` and `plan_california_repair_v1`, complete road-valid local OSRM context, implement `plan_diff` and independent evaluation, and then add one ownership-aware single-day sequential repair. Do not start with a full conversational agent, nationwide graph, or learned scenic perception model.

The project now has enough data, optimization, visualization, and Phase 0 evidence infrastructure. Its next weakness is not lack of features; it is the absence of one authoritative contract connecting a traveler-approved plan, the semantics of edits, the optimizer’s smallest repair, the final routed artifact, the independent certificate, and the explanation. Implementing and evaluating that contract is the shortest path to a defensible publication.

Appendix A. Mathematical Notation Summary

Symbol	Definition
π_0 / π	Parent and child itinerary.
ω	Versioned disruption snapshot.
N_r	Editable neighborhood at repair radius r .
x_{id}	POI-day assignment.
y_{ijd}	Within-day route arc.
z_{hd}	Lodging after day d .
δ^- / δ^+	Deletion and addition indicators.
m_{id}	Day-move indicator.
η_{hd}	Hotel-change indicator.
r_{ijd}	Road-change indicator.
ξ_c	Constraint-relaxation variable.
L0-L3	Sequential lexicographic objectives.
σ	Solver and independent evaluation certificate.

Appendix B. Suggested Public Schemas

```

@dataclass(frozen=True)
class OwnedConstraint:
    constraint_id: str
    origin: ConstraintOrigin
    strength: ConstraintStrength
    scope: ConstraintScope
    target_id: str
    confirmed: bool
    relaxation_policy: RelaxationPolicy
    evidence_refs: tuple[str, ...] = ()

@dataclass(frozen=True)
class RepairRequest:
    repair_request_id: str
    parent_plan_id: str
    disruption_snapshot_id: str
    edits: tuple[EditRequest, ...]
    allowed_radii: tuple[RepairRadius, ...]
    test_only: bool = False

@dataclass(frozen=True)
class RepairOutcome:
    status: RepairStatus
    parent_plan_id: str
    child_plan_id: str | None
    selected_radius: RepairRadius | None
    planner_run_ids: tuple[str, ...]

```

```
plan_diff_id: str | None
evaluation_id: str | None
```

Appendix C. Current Main-Branch Audit Anchors

Path	Audited blob SHA	Finding
README.md	303457951ce7e0daff833c6b5765aa939aebd49a	Notebook remains documented entry point; static-dashboard and proxy limitations are explicit.
src/itinerary_system/config.py	846197d42dc9e91b782da19052997c9a6111daf1	Snapshot/run identifiers already exist; repair/provider/routing policy needs expansion.
src/itinerary_system/data/snapshot.py	c024aae14c58f102a92e5f0937f179fdd7c43f69	Clean-clone snapshot loader/validator landed.
src/itinerary_system/routing/models.py	8010f1039a1dbc511f0d82099a4f604d9eb4bc2a	Route provenance and evaluation eligibility landed.
src/itinerary_system/research_artifacts.py	69eaab6a32f04e651ef921fc5bdc83e2d0a6f003	PlannerRun, PlanArtifact, and evaluation contracts landed.
src/itinerary_system/multi_objective_route.py	555470af8aa6d64a6dd2fab4ae131b486fcfea62	Current single-day route solver is not parent-aware repair.
src/itinerary_system/route_gurobi_oracle.py	74c86ad4e4ca8637e0294415f110d645c1302526	Early return leaves unreachable legacy implementation.
src/itinerary_system/experiment_runner.py	adf1f092cc6d0e5b0285b77bb7ac52ee25847d86	Still notebook-oriented and uses flat production artifacts.
src/itinerary_system/map_renderer.py	151718240f878b38de122e2ba06a55827b356052	Compatibility wrapper actively strips stale notebook state.
docs/research_question_and_phase0_execution.md	856ef4a49955294832ba8c4f790565a3a4f45809	Phase 0 direction and strict road-validation gates documented.

Appendix D. Lodging Provider Matrix

Provider	Role	Activation	Research policy
OSM/Overpass	Stable identity, coordinates, broad accommodation type.	No commercial key; cache/freeze query.	Required benchmark base with ODbL attribution.
Geoapify	Optional place/category/geocoding enrichment.	API key; free plan currently available.	Optional; cache with attribution and query manifest.
LiteAPI	Date-specific rate and availability.	Sandbox/production key; probe before use.	Optional context snapshot; sandbox clearly labeled.
Google Places	Rating, count, status, address, website, photos.	Billing-enabled key and field-specific requests.	Optional metadata only; never benchmark dependency.

Provider	Role	Activation	Research policy
Booking.com Demand API	Partner accommodation/rate data.	Commercial partner credentials.	Dormant adapter until contract and valid key.
Expedia Rapid	Lodging and vacation-rental supply/rates.	Partner onboarding and valid credentials.	Dormant adapter until approved.
Airbnb	Provider-specific rental inventory.	Official partner/program access only.	No scraping; omit direct adapter without access.

Appendix E. Minimum Test Matrix

Area	Required test
Snapshot	Clean clone loads california_v1 and manifest hashes match.
Provenance	Every optimizer feature has evidence classification and transformation version.
Lodging	Canonical category mapping is deterministic; provider brand never inferred from type alone.
Provider gate	Missing/invalid credential produces UnavailableProvider and no hidden network call.
Routing	Geodesic fallback is never road_validated or final-comparison eligible.
Local OSRM	Pinned endpoint returns complete matrix and selected-route geometry for benchmark legs.
Parent immutability	Repair never mutates parent content hash.
Ownership	Locked items cannot change; booked relaxations require explicit variable and confirmation.
Lexicographic	Later objectives never degrade fixed earlier optimum beyond tolerance.
Neighborhood	Uneditable days remain byte-equivalent or semantically identical.
Plan diff	Adds/deletes/day moves/hotel/road/order changes match independent recomputation.
Certificate	Post-solve mutation invalidates certificate until a new evaluation.
Explanation	Every numerical/causal statement has valid evidence references.
Benchmark split	No parent-plan/disruption family crosses development and held-out test.

References

- [1] Ztang, Y. X. Publication-Oriented Research Roadmap: From Weather-Aware Route Generation to Inspectable, User-Controlled, Solver-Verified Itinerary Repair. 2026.

- [2] Ztang, Y. X. An Inspectable and User-Controlled System for Weather-Aware Itinerary Repair. Research Direction Report. 2026.
- [3] Ztang, Y. X., Wang, X., and Zhang, L. Weather-Aware Utility Maximization for Urban Attraction Planning. IE 5533 Final Project Report. 2026.
- [4] Karmakar et al. TripTide: A Benchmark for Adaptive Travel Planning under Disruptions. Findings of ACL 2026.
- [5] Huang et al. iTIMO: An LLM-empowered Synthesis Dataset for Travel Itinerary Modification. 2026.
- [6] Ju et al. To the Globe: Towards Language-Driven Guaranteed Travel Planning. EMNLP System Demonstrations, 2024.
- [7] Chen et al. OptiChat: Bridging Optimization Models and Practitioners with Large Language Models. INFORMS Journal on Data Science, 2025.
- [8] Otto, Kurtz, and Birbil. Coherent Local Explanations for Mathematical Optimization. 2025.
- [9] Quercia, Schifanella, and Aiello. The Shortest Path to Happiness. 2014.
- [10] OpenStreetMap Foundation. Copyright and License; ODbL attribution requirements.
- [11] Open-Meteo. License, Terms, API and Attribution Documentation.
- [12] U.S. National Park Service. Developer Resources and NPS Data API.
- [13] Project OSRM. HTTP API documentation for Table, Route, and Nearest services.
- [14] OSMnx documentation. Routing and k-shortest-path functions on MultiDiGraph road networks.
- [15] LiteAPI. Hotel rates endpoint, sandbox-key, and booking documentation.
- [16] Geoapify. Places/Geocoding APIs and current pricing documentation.
- [17] Google Maps Platform. Places API field types, place types, pricing, billing, and usage limits.
- [18] Booking.com Demand API. Partner prerequisites and accommodation type documentation.
- [19] Expedia Group Rapid API. Lodging/vacation-rental supply and partner requirements.
- [20] Airbnb Terms and official partner/API program documentation.
- [21] ACM IUI 2027. Conference and paper deadlines.
- [22] ACM CHI 2027. Papers call and full-paper deadline.
- [23] Current GitHub repository main branch and audited files listed in Appendix C.